Numerical Analysis Assignment 1: Round-Off and Truncation Error

Addison Olstad

October 2026

1 Introduction

Computers are powerful tools for large mathematical computations, but for two common numerical computations, differentiation and quadrature, there exist round-off and truncation errors. Differentiation is the process of estimated a function's derivative using discrete function values, while quadrature is the approximate computation of a definite integral using a weighted finite sum of function evaluations.

Truncation error arises when derivatives and integrals are replaced by finite approximations. The Round-off error arises because computers store and calculate numbers with finite precision. In numerical differentiation, subtracting nearly equal function values can amplify these rounding errors.

2 Methods

Considering the test function:

$$u(x) = e^{sin(x)}$$

on a regular grid spanning [-π, π) with N points. Python can be used to evaluate the function and its derivative on the grid, as well as to apply a finite-difference scheme to approximate u' from the values of the grid points of u on the same grid. In addition, compute an error for the grid of N points and repeat the computation to increase N and plot the error versus N.

Algorithm 1 Finite-difference errors on a periodic grid

Define $u(x) = e^{\sin x}$ and $u'(x) = e^{\sin x}\cos x$
for each grid size N **do**
    $h \leftarrow 2\pi/N$
    $x_j \leftarrow -\pi + jh$, for $j = 0, \ldots, N-1$
    $U_j \leftarrow u(x_j)$
    for each finite-difference scheme **do**
        Compute $D_h U_j$ using periodic neighboring indices
        $E_N \leftarrow \max_j |D_h U_j - u'(x_j)|$
        Record N and E_N
    end for
end for
Plot E_N versus N for each scheme

Consider the initial value problem:

$$\dot{u} = \frac{1}{1+u}$$

for

$$u(0) = 1$$

To find the exact solution, separate the variables

$$(1+u)du = dt$$

Integrate both sides:

$$u + \frac{u^2}{2} = t + C$$

Then use the initial condition, $u(0) = 1$ to find C

$$1 + \frac{1}{2} = C = \frac{3}{2}$$

Therefore after simplifying and solving for u:

$$\boxed{u(t) = \sqrt{4 + 2t} - 1}$$

A computational approach to this method is called the Picard iteration. It is an iterative method that approximates the equations solution. The given approximate solution is mapped to a new approximation through:

$$\boxed{u^{(k+1)}(t) = u(0) + \int_{t'=0}^{t} \frac{1}{1 + u^{(k)}(t')} dt'}$$

Algorithm 2 Grid-based Picard iteration

Choose $t_1 = 1$ and increasing grid sizes N
for each N, while within the computation budget **do**
    $t_j \leftarrow jt_1/(N-1)$, for $j = 0, \ldots, N-1$
    $U_j \leftarrow 1$
    $e_N \leftarrow \max_j |U_j - u(t_j)|$
    loop
        Compute U_{new} using cumulative trapezoidal integration:

$$U_{\text{new},j} = 1 + Q_j\left(\frac{1}{1+U}\right)$$

        $e_{\text{new}} \leftarrow \max_j |U_{\text{new},j} - u(t_j)|$
        if $e_{\text{new}} \geq e_N$ **then**
            break
        end if
        $e_N \leftarrow e_{\text{new}}$
        $U \leftarrow U_{\text{new}}$
    end loop
    Record N and e_N
end for
Plot e_N versus N

3 Results

3.1 Test Function

After writing up the pseudocode in Python for the finite differencing method, all three finite difference schemes (forward, backward, central) were tested and plotted. Each resembles a V-shaped curve, with different minimums and slopes, although the forward and backward curves almost overlap.

The slopes with respect to step size h were approximately 1.9991 for central differences and 0.9994 for both forward and backward differences. All agree with theoretical truncation errors of $O(h^2)$, $O(h)$, and $O(h)$. Since the plot uses N rather than h, the corresponding slopes are approximately -1.9991, -0.9994, and -0.9994. Therefore, in the truncation-dominated region, halving the step size reduces the central scheme error by about four and the forward and backward errors by about two.

The best tested step size depended on the scheme. Central differences had a minimum maximum absolute error of 1.27×10^{-10} at $h \approx 1.20 \times 10^{-5}$, with $N = 524{,}288$. Forward and backward differences had minimum maximum absolute errors of 8.08×10^{-8} and 8.04×10^{-8}, respectively. Both occurred at $h \approx 4.68 \times 10^{-8}$, with $N = 134{,}217{,}728$.

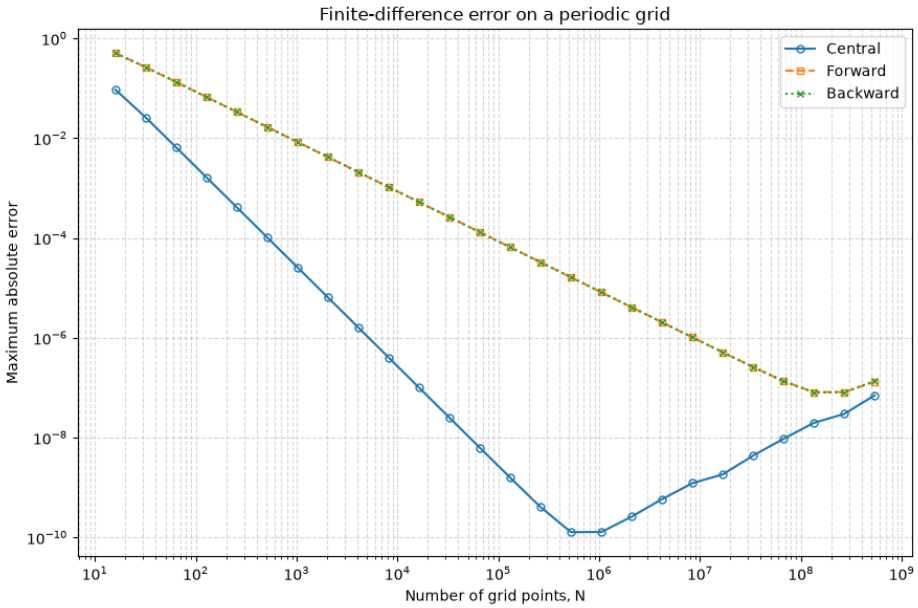


Figure 1: Maximum absolute error versus the number of grid points N for all three finite difference schemes.

Therefore, central differences had an error approximately 635 times smaller while using a larger step size. The forward and backward curves almost overlap because their leading truncation errors have opposite signs but similar magnitudes, and the function's symmetry gives nearly equal maximum absolute errors. For small enough step sizes, h, subtracting nearly equal function values exposes floating-point rounding errors, which division by h amplifies. Further refinement of the step size eventually causes the error to fluctuate and increase.

3.2 Initial Value Problem

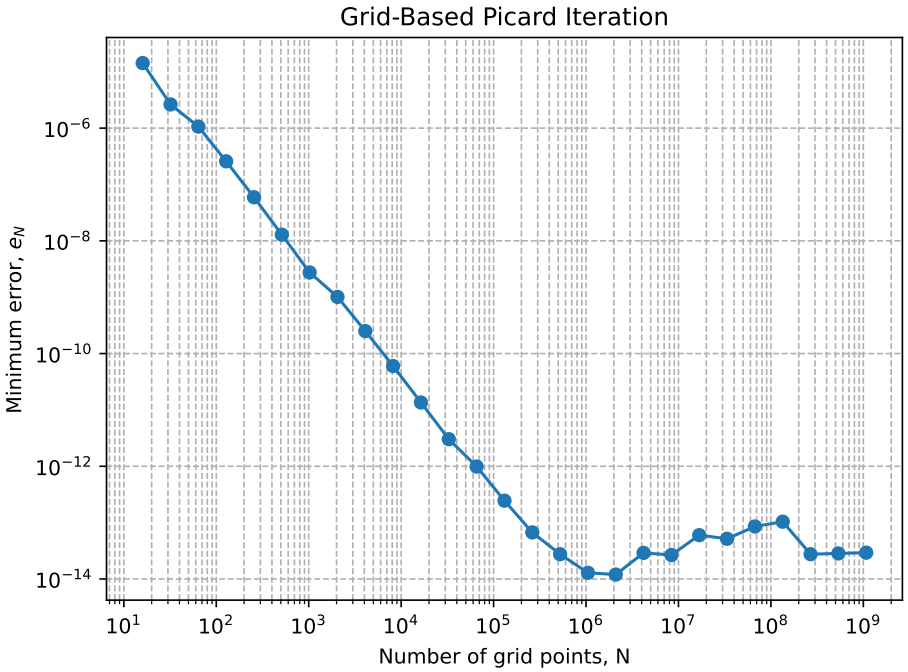


Figure 2: Picard Iterations: Minimum maximum absolute error e_N versus the number of grid points N.

For $t_1 = 1$, the minimum Picard iteration error initially decreases approximately as N^{-2}. The slope was approximately found through a log-log fit over $64 \leq N \leq 131{,}072$, which returned a slope of -2.0011 that is consistent with the $O(h^2)$ global error of the trapezodial rule.

Small deviations from this straight line can be seen on the right side of the V-shaped plot. This arises because the iteration changes with N as well as the iteration and quadrature errors can still partially cancel. The smallest error recorded was 1.20×10^{-14}, obtained with $N = 2{,}097{,}152$ points. All in all, further refinement produced fluctuations rather than consistent improvement, indicating that floating-point round-off had become significant. The largest completed grid contained 1,073,741,824 points, but its error was approximately 2.93×10^{-14}, larger than the minimum.

4 Conclusion

In conclusion, both experiments show that refining the grid initially reduces truncation error, but eventually round-off limits further improvement. Central differences achieved greater accuracy than forward and backward differences, while the Picard results agreed with the second-order accuracy of the trapezoidal rule. Therefore, a finer grid does not always produce a more accurate result.